EXPERIMENT PRESENTATION

# File Transfer Service

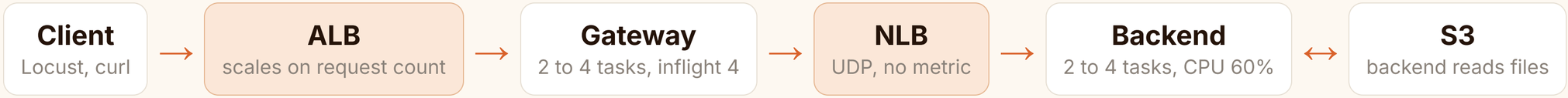
An empirical evaluation on AWS. Three experiments on packet loss, file size, and load and autoscaling.

**Amulya Penikalapati**
CS6650 Building Scalable Distributed Systems, Midterm Mastery Part 2

THE SERVICE

# How the service works

A client asks for a file over HTTP. The gateway fetches it from the backend over UDP in 64 byte chunks, up to about 4 MB. Replies are 200 ok, 404 not found, 503 busy (no free transfer slot on the gateway), or 504 (the backend never finished the transfer).




**Why these three experiments.** Loss, size, and load probe protocol reliability, protocol scalability, and deployment scalability. I hold the retries (5 tries, 5 rounds) and max inflight (4) fixed and vary one thing at a time. On AWS the gateway and backend autoscale on **different signals**, which Experiment 3 shows really matters.

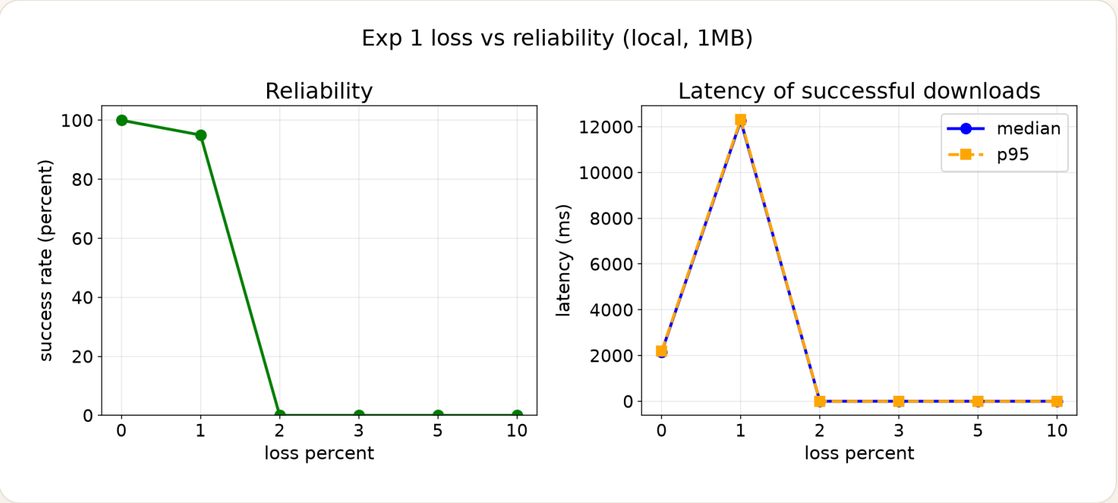
EXPERIMENT 1 LOCAL

# Packet loss vs reliability

- **Vary** backend loss 0, 1, 2, 3, 5, 10 percent
- **Fixed** 1 MB file, default retries, one request at a time, 20 downloads each

| loss | success | median latency |
|---|---|---|
| 0 percent | **100 percent** | 2.15 s |
| 1 percent | **95 percent** | 12.3 s |
| 2 percent | **0 percent** | all 504 |
| 3, 5, 10 percent | **0 percent** | all 504 |

The 1 MB file falls off a **cliff between 1 and 2 percent** loss. At 1 percent it still works, but latency jumps almost 6 times as the client keeps asking for dropped chunks. At 2 percent and up it never completes.




success drops to 0 above 1 percent loss, the right panel is latency of the ones that succeed

EXPERIMENT 2 LOCAL + AWS

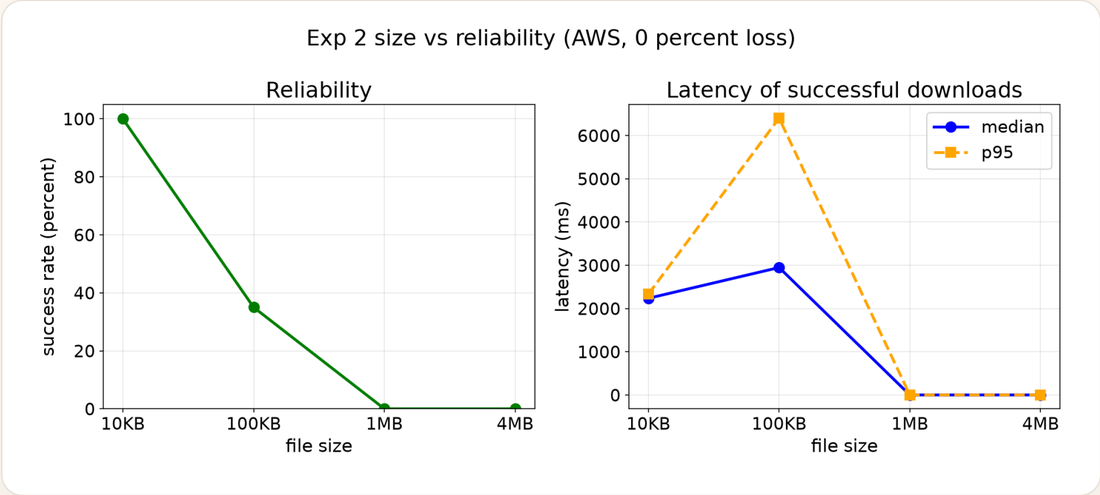
# File size vs reliability

- **Vary** file size 10 KB, 100 KB, 1 MB, 4 MB
- **Fixed** local at 10 percent loss, AWS at 0 percent configured loss, 20 downloads each

**Hypothesis** bigger files fail more than their size alone suggests, since more chunks means more chances one is lost.

| size | local, 10% loss | AWS, 0% loss |
|---|---|---|
| 10 KB | **100%** | **100%** |
| 100 KB | 70% | **35%** |
| 1 MB | **0%** | **0%** |
| 4 MB | **0%** | **0%** |

Confirmed. And **AWS at 0 percent loss is worse than local at 10 percent**. 0 percent is the fair AWS baseline, because the real network already adds its own loss, and it still loses to 10 percent injected locally.




AWS run, success falls as files grow even at 0 percent loss (local numbers in the table)

EXPERIMENT 3 AWS

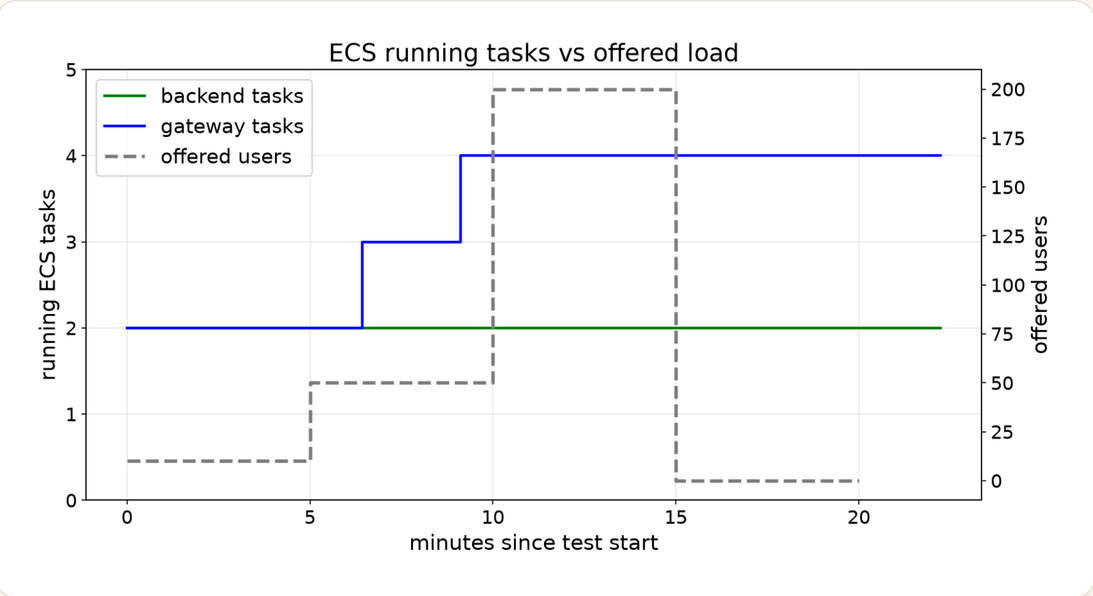
# Load and autoscaling on AWS

- **Vary** Locust load 10, 50, 200, 0 users, 5 min each
- **Fixed** 10 KB file, 0 percent loss, deploy defaults

14,364 requests | **63% failed, all 503** | p95 7.1 s

**gateway 2 → 4 tasks** | backend stays 2 | ECS CPU ~3%

The gateway scales on request count and grows 2 to 4. The backend scales on CPU, but a UDP server is not CPU bound, so CPU sits near 3 percent and it **never scales**. Even at 4 gateway tasks, max inflight 4 means most of 200 users are shed with 503. **CPU is the wrong scaling signal** for this backend.




gateway scales 2 to 4, backend stays at 2, as the load rises




Locust throughput and failures, same run

AWS METRICS

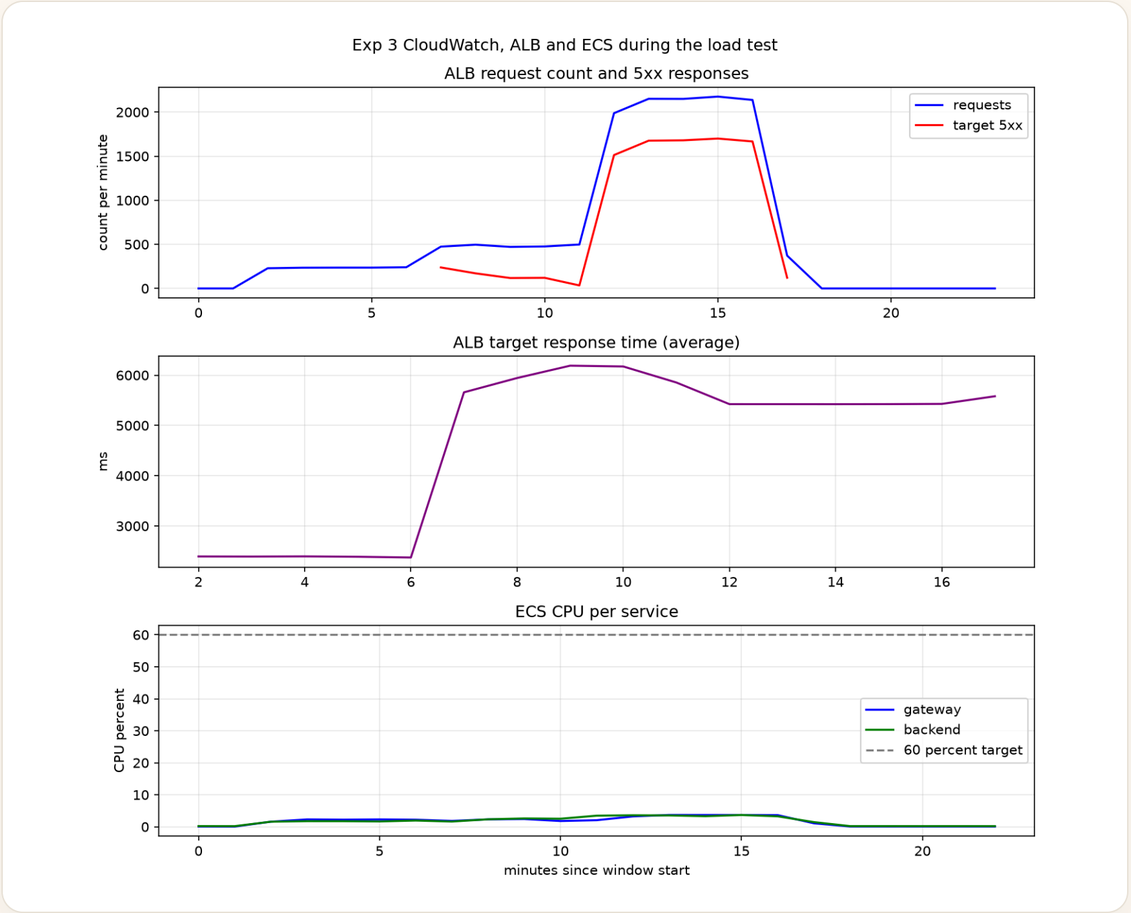
CLOUDWATCH

# What CloudWatch showed

I pulled the AWS side metrics from CloudWatch to confirm, from the infrastructure view, what the service did during the load test.

- the ALB saw about 2,100 requests per minute at peak, about 1,700 of them 5xx (the 503 shedding)
- target response time rose from about 2.3 s to about 6 s under load
- ECS CPU stayed near 3 percent for both services, far below the 60 percent target

> This is why the backend never scaled. Its scale out alarm watches **CPU**, and CPU never came close to 60 percent.




ALB requests and 5xx, response time, and ECS CPU with the 60 percent target line

WHAT THIS REVEALS

# Three ways it breaks

1 **It cannot tolerate loss on large files.** The chunked UDP protocol with a fixed retry budget stops converging once more than a few chunks go missing.

2 **It scales on the wrong signal.** The backend scales on CPU but is never CPU bound, so it never scales where the real work is.

3 **It sheds load early.** The gateway allows only four transfers in flight per task, so it returns 503 well before it is truly saturated. And the AWS network behaves like a lossy link even with no configured loss.

LIMITATIONS AND NEXT STEPS

# What I would test next

## Limitations

- 20 downloads per level shows the cliffs but does not put error bars on the edges
- Experiment 3 effectively ended near 15 minutes, and I did not wait for the slow scale in

## What I would test next

- Finer loss levels around 1 to 2 percent, with more repeats
- Raise the gateway max inflight and run the load test again
- Change the backend scaling signal from CPU to active transfers or NLB flow count

All data, charts, and code are in the repo under the **midterm mastery** folder.